The FL-IDS website, explained simply

A walk through every section of **aronnnn1.github.io/FL-IDS** and its **Cumulative results** page: what each line says, and what it means in plain words.

How to use this: each part shows what the site says (grey), then a simple explanation (yellow), and sometimes why it matters (green) or a warning (pink). A dictionary of every attack type and every score is at the end.

The story we use throughout

Like you're five: Think of a big school with many classrooms. Children pass little **notes** to each other all day; these are the **packets** travelling on a network. Most notes are friendly (**normal traffic**), but some come from **bullies** (**attacks**). Our system is a team of helpers: **guard 1** has a photo book of 14 known kinds of bully (**LightGBM classifier**); **guard 2** knows what normal notes look like and spots anything weird (**autoencoder**); the **classrooms learn together** by sharing lessons, not notes (**federated learning**); a **fair principal** throws out lessons from sneaky classrooms (**trust filter**); and an **automatic door** locks bullies out (**SDN**).

What's in this guide

1. Top bar and title
2. The five headline tiles
3. Boosting vs. a deep-learning classifier
4. Confusion matrices
5. Training time
6. How macro F1 was pushed past 0.95
7. The pipeline at a glance
8. Every step explained (12 steps)
9. Does the pipeline work?
10. Caveats and limitations
11. Cumulative results page (sections 1–8 + index)
12. Dictionary: the 15 kinds of traffic
13. Dictionary: every score and word

1. The top bar and the title

On the site: Menu links: Summary · Boosting vs. DL · Confusion matrices · Training time · Accuracy ≥ 95% · Pipeline · Step-by-step · Verification · Caveats · Cumulative results → and a Light/Dark mode button.

Like you're five: This is the table of contents. Clicking a word jumps to that part of the page. The button changes the colours from white to dark, like turning the lights down. “Cumulative results” opens a second page with every table of numbers in one place.

On the site: Title: “Boosting vs. Deep Learning, and the full FL-IDS pipeline”.

Like you're five: The page tells two stories: a *contest* between two kinds of guard 1 (a “boosting” guard and a “deep-learning” guard), and the *whole system* from start to finish (the “pipeline”, like a factory line).

On the site: “A federated, cascade-based intrusion detection system for IoT/IIoT traffic: a supervised LightGBM classifier as the first pass, with a client-side autoencoder as a federated, poisoning-resistant backstop for zero-day attacks.”

Like you're five: Word by word: **intrusion detection system** = a burglar alarm for networks. **IoT/IIoT traffic** = notes passed between smart gadgets in homes and factories. **federated** = many classrooms learn together without handing over their notes. **cascade** = a line of checks, one after another, like two guards at two gates. **supervised LightGBM classifier as the first pass** = guard 1, who learned from labelled examples (“this note is from bully X”). **client-side autoencoder** = guard 2, who lives in each classroom. **poisoning-resistant** = hard to trick during training. **backstop for zero-day attacks** = a safety net for bullies nobody has ever seen.

On the site: “This site reports (1) a head-to-head comparison of boosting against a deep-learning classifier, (2) how long things take and why, (3) how the classifier was pushed past 0.95 macro F1 and 95% accuracy — and checked on a stricter, connection-held-out test, (4) a visual and step-by-step explanation of the whole pipeline, and (5) verification that it works end to end, with its limitations stated plainly.”

Like you're five: The five things you'll find: (1) which kind of guard 1 is better; (2) how long the computer took for each job; (3) how we got the score above our 95% goal, and proved we didn't cheat by testing on whole conversations the system had never seen; (4) pictures and steps showing how everything works; (5) proof that it all works together, plus what it still can't do.

On the site: “Dataset: Edge-IIoTset (Ferrag et al., 2022). All numbers come from real runs of the code in this repository. Full notes and sources: PAPER_NOTES.md.”

Like you're five: The notes we learned from come from a public collection called **Edge-IIoTset**, recorded by researchers (Ferrag and team) on a real test network with more than ten kinds of sensors. Every number on the site came from actually running our programs; nothing was made up. PAPER_NOTES.md is our diary of every experiment.

On the site: “Project status: complete (16 October 2026). Every phase is implemented, run on the real data and verified end to end; the research paper is being written.”

Like you're five: The project is finished: every part was built, tried on the real notes and checked from start to end. The only thing left is writing it up as a research paper.

2. The five headline tiles

Tile	What it says	Like you're five
Final classifier macro F1	0.983; 99.7% accuracy on held-out connections (was 0.889 / 96.4%)	Guard 1's final report card. 0.983 out of 1 means it is excellent at *every* kind of bully, even rare ones. It gets 99.7 of every 100 notes right, on conversations it had never seen. Before our improvements it was 0.889 and 96.4%.

Boosting vs. DL	0.887 macro F1 vs. 0.622 for a comparable DL classifier (base features)	In the contest, the boosting guard scored 0.887 and the deep-learning guard only 0.622. Boosting won by a lot. (This contest used the original clues, before our improvements.)
Cascade zero-day recall	0.943 vs. 0.801 for boosting alone (was 0.731 / 0.518)	When we hid a kind of bully from training and then tested, both guards together caught 94 of every 100; guard 1 alone caught 80. Before the new clues it was 73 and 52.
Real Phase A run	0.999 attack macro-recall, real 10-client FL training (was 0.889)	We ran the real thing with ten separate programs acting as ten classrooms. It caught about 99.9% of bullies, averaged over bully types (89% before the new clues).
Real SDN demo	Correct: Backdoor attack classified & blocked live	We replayed a real attack; the system named it correctly ("Backdoor") and the automatic door locked the attacker out.

3. Boosting vs. a supervised deep-learning classifier

On the site: "If you swap LightGBM boosting for a deep-learning classifier as the cascade's first pass — same attack-type labels, same task, comparable effort — which is more accurate? Both were trained and evaluated on the exact same 500,000-row sample (400k train / 100k test, stratified); boosting on raw features, the DL classifier on standardized features."

Like you're five: A fair race between two guards. **Boosting** (LightGBM) is a team of hundreds of tiny yes/no questions ("Is the note longer than 100 letters? Is it sent to door 80?") whose votes are added up. **Deep learning** (a small neural network, an "MLP" with 8,271 adjustable knobs) learns by mixing all the numbers together through layers. Both got the same 500,000 notes: 400,000 to study and 100,000 for the exam. "Stratified" means each bully type appears in study and exam in the same proportions. "Standardized" means the deep-learning guard got the numbers rescaled to similar sizes, because that's what it needs; the boosting guard doesn't need that.

The comparison table

Row	Boosting	DL	What it means
Accuracy	0.8908	0.6779	Share of exam notes answered right: 89% vs 68%.
Macro F1	0.8872	0.6218	Average score over the 15 note types, each counted equally. The fairest single number.
Weighted F1	0.8924	0.6681	Same, but big note types count more.
Benign false-positive rate	0.1599	0.5341	How often a *friendly* note is wrongly called a bully. Lower is better: 16% vs 53%.
Train time	55.5 s	10.2 s	Study time. Deep learning studied faster but learned less.
Model size	12.8 MB, 300 trees × 15 classes	8,271 parameters	How big each "brain" is.

The "Gap" column is boosting minus DL; a plus sign means boosting is better.

“Per-class recall: where the gap actually is”

On the site: Bars for each of the 15 classes, Boost vs DL, e.g. XSS 0.761 vs 0.072, SQL_injection 0.767 vs 0.183, Fingerprinting 0.579 vs 0.000, Port_Scanning 0.997 vs 0.498 … DDoS_TCP/UDP 1.000 vs 1.000, Password 0.831 vs 0.860.

Like you're five: **Recall** = “of all the bullies of this type, how many did you catch?” Each pair of bars is one bully type. Boosting catches 76% of XSS bullies; deep learning catches only 7%. For floods (DDoS) both catch everything, because floods are loud and obvious. The bars are sorted so the biggest differences come first. Password is the one type where DL is slightly better (0.860 vs 0.831), but only because DL guesses “Password” for almost everything (see the confusion matrices).

On the site: “Why boosting wins here.” Trees split directly on the numbers (packet counts, flags, protocol fields); a small network must learn the same boundaries through weighted sums, and with 400k rows and 15 epochs it doesn't manage it for subtle classes.

Like you're five: Our data is a big spreadsheet of numbers. Boosting asks sharp questions like “is this number above 5?”, which is exactly how spreadsheet data separates. The small network has to blend everything and learn slowly; with 15 practice passes (“epochs”) it never learned the fine differences between sneaky web attacks and normal traffic.

On the site: “Honest caveat.” The DL model was deliberately small; a bigger one might do better. The right claim is “this DL classifier doesn't beat boosting here”, not “DL cannot work”.

Like you're five: We only raced against a small deep-learning guard. A bigger one, trained longer, might do better, and we say so instead of overclaiming.

4. Confusion matrices

On the site: “Rows are the true class, columns the predicted class. Cell colour is the fraction of that true class landing in that column, so the outlined diagonal reads as recall. Hover a cell for the exact count.” Buttons: **Counts** / **% of true class**.

Like you're five: A confusion matrix is a **scoreboard of mix-ups**. Each **row** is what the note *really* was; each **column** is what the guard *said* it was. The diagonal (top-left to bottom-right) is where the guard was right. Anything off the diagonal is a mistake: row “Normal”, column “Password” = friendly notes the guard wrongly called password attacks. Darker colour = more notes. “Counts” shows how many notes; “% of true class” shows what share of that row. The colour bar shows 0% to 100% of the row.

Boosting's matrix (500k comparison, original clues)

Like you're five: Mostly a strong diagonal, with some mix-ups among web traffic. Examples you can read off the table: of 9,848 real **Normal** notes, 8,273 were called Normal, but 716 were called Password, 521 SQL_injection and 325 DDoS_HTTP. Of 7,361 **Uploading** notes, only 4,634 were right; 1,443 were called DDoS_HTTP and 553 Port_Scanning. Floods (DDoS_TCP, DDoS_UDP: 9,848 each) are perfect.

On the site: “Boosting's errors sit mostly inside one application-layer cluster — DDoS_HTTP, Password, SQL_injection, Uploading, XSS and Normal — which together account for 84% (9,225 of 10,921) of all its mistakes.”

Like you're five: Of all 10,921 mistakes, 84% were confusions *within* one family of six web-related types. These all look alike because they all talk to the same web server. That told us exactly where to improve.

Deep learning's matrix

On the site: “Password absorbs large fractions of DDoS_HTTP, Normal, SQL_injection, Uploading and Vulnerability_scanner — a strong ‘when unsure, guess Password’ bias … The same cluster holds 78% (25,224 of 32,207) of its mistakes, but it also fails on Port_Scanning ↔ DDoS_TCP and Fingerprinting.”

Like you're five: The deep-learning guard made three times as many mistakes (32,207), and had a bad habit: when unsure, it shouted “Password!”. Of 9,848 Normal notes it called 4,532 Password, and of 3,013 XSS notes it called 1,981 Password. It also called half of the Port_Scanning notes (1,990 of 3,995) DDoS_TCP, and never once got Fingerprinting right.

Final classifier's matrix (after our improvements)

On the site: Boosting with the new clues and class weights, on all 1,985,768 rows: trained on 1,429,825 and tested on 397,073 from whole connections never seen in training. “The HTTP-family cluster holds 67% (744 of 1,112 remaining errors).” Remaining mix-ups: Password ↔ DDoS_HTTP, Ransomware → Backdoor. “MITM has no row here … on the random split its F1 is 0.994.”

Like you're five: This exam was much bigger (about 397,000 notes), and the guard made only **1,112 mistakes** in total. Examples: of 287,319 Normal notes, 287,082 were right; of 10,197 SQL_injection notes, 10,191 were right. Remaining mix-ups: 406 Password notes called DDoS_HTTP and 198 the other way (both knock on the same website), and 98 Ransomware notes called Backdoor (both use door number 4321). **MITM** is missing because all of its notes are *one single conversation*, which can't be split between studying and the exam. On the normal (random) exam MITM scores 0.994.

The final classifier's mix-up scoreboard as shares of each row. Dark diagonal = right.

5. Training time: how long, and why

On the site: “Training either classifier is quick (boosting ~1 minute, DL ~10 seconds). What is slow is the evaluation methodology — the poisoning sweep and zero-day experiments retrain the whole federated pipeline 75 and 14 times.”

Like you're five: Teaching one guard is fast. What takes hours is our *testing*: to prove things, we rebuild the whole school's training from scratch again and again (75 times for one test, 14 for another), like rehearsing a whole play 75 times.

On the site: Bar: Boosting 55.5 s, “5.4× slower”; DL classifier 10.2 s, “baseline”.

“Measured wall-clock times” table, row by row

Job	Time	Simple meaning
Boosting classifier (comparison)	55.5 s	300 rounds × 15 types = 4,500 tiny trees, on purpose on one CPU core so results repeat exactly.
Final classifier	24 s	Our new guard 1 on 1.43 million notes. It stopped early after 54 rounds once it stopped improving, using 4 cores. That's a quicker exploration setting, not the exact-repeat mode.
Building the new features	≈ 1.5 min	Reading all 2.2 million notes, opening their contents and grouping them into about 393,000 conversations. Done once and saved (as a “Parquet” file).
DL classifier	10.2 s	15 passes over the data in batches of 256 notes; a tiny brain (8,271 knobs).
One in-process federated run	≈ 2 min	20 rounds of classrooms learning together inside one program; about 3 s per round, plus 20–25 s to read the 1.16 GB file.
Real Phase A	≈ 4 min [5 min 15 s]	The real version: 1 principal program + 10 classroom programs talking over the network, including 6 guard-1 updates. Faster now because the 130 clues are read from a saved copy instead of re-reading the big file.
Standalone zero-day	≈ 46 min [≈ 61 min]	14 full retrainings (one per hidden bully type), about 3.3 min each.
Ablation	≈ 2 h 40 min	5 versions of the system, plus 70 hidden-bully retrainings.
Poisoning sweep	≈ 3 h 45 min [≈ 5 h]	75 full retrainings at different numbers of sneaky classrooms.
Full test suite	≈ 7.5 min [6 min 14 s]	233 automatic checks that the code does what it should (224 before, plus 9 for the new clues).

On the site: “VM time = time on the Linux VM's own clock; the VM pauses whenever the Mac sleeps … single measurements, not averaged.”

Like you're five: Our programs ran inside a pretend computer (a **virtual machine**) on a MacBook. When the Mac slept, the pretend computer's clock paused too, so these times are “awake time”. Each was measured once.

“Why a full federated run takes ~4 minutes when a round takes seconds”

- **Boosting updates are the bottleneck.** Every 3 rounds guard 1 gets extra lessons. That round takes about 43 s instead of 5–7 s: classrooms spend about 10 s checking notes with the bigger guard, and the principal spends about 33 s teaching it 50 more trees per type. Six of these per run take about three-quarters of the time (estimated from timestamps).
- **Single-threaded on purpose.** If guard 1 studies with several cores at once, tiny timing differences change the result. One bully type's score jumped between 0.0 and 0.88 on identical runs! So we force one core: slower, but repeatable.
- **The model grows.** Each update adds trees (300 → 600), so later rounds are slower.
- **Evaluation multiplies everything.** Every test point is a full retraining, so minutes become hours.
- **Modest hardware.** No graphics card (not needed: our brains are tiny), 4 cores, limited memory: 11 programs loading big libraries nearly ran out until we limited their helper threads.
- **Data loading every time.** Reading the 1.16 GB file takes 15–25 s per program start.

On the site: “How to speed it up (none applied yet)”: cache the cleaned data, update guard 1 less often, allow multi-core for exploring, use a machine with more memory.

Like you're five: Ideas to make it faster that we haven't done yet: save the cleaned notes so we don't re-read them, give guard 1 fewer extra lessons, use several cores when we're just exploring, and get a computer with more memory.

6. How macro F1 was pushed past 0.95

On the site: “The target was ≥ 95% accuracy and ≥ 0.95 macro F1. With the original 91 features, boosting reached 0.887 macro F1 on the 500k comparison sample, and 0.889 macro F1 / 96.4% accuracy on all 1.9M rows. Ideas 1, 2, 7 and 8 below were implemented and measured … 0.983 macro F1 and 99.7% accuracy … Ideas 3–6 turned out not to be needed.”

Like you're five: Our goal: at least 95 right out of 100, and a fairness score (macro F1) of at least 0.95. With the original 91 clues per note we had 96.4% but only 0.889 fairness, because some bully types were badly caught. We tried four ideas; they got us to 99.7% and 0.983. Four other ideas were never needed.

“Before: the F1 budget of the original features”

On the site: “Macro F1 is the plain average of 15 per-class F1 scores, so reaching 0.95 needed the per-class F1 sum to rise from 13.31 (= 0.887 × 15) to 14.25 — a gain of 0.94 F1 points.” Bars: Fingerprinting F1 0.683 (−0.267), Uploading 0.761, DDoS_HTTP 0.762, SQL_injection 0.768, Password 0.787, XSS 0.857, Normal 0.905, Port_Scanning 0.916, Ransomware 0.933, then Backdoor 0.962 and the rest “ok”.

Like you're five: Imagine 15 school subjects with marks out of 1. The average must reach 0.95, so all marks together must add up to 14.25. We had 13.31, so we were 0.94 marks short. The red bit of each bar shows how far each subject was below 0.95. The weakest “subjects” were Fingerprinting, Uploading, DDoS_HTTP, SQL_injection and Password.

“Why the original features were stuck”

On the site: “In SQL_injection, XSS, Uploading and Password traffic only about 5–17% of packets carry any HTTP content. The rest are bare TCP handshake/acknowledgement packets to port 80 … 2.31% of rows share an identical 91-feature vector with a row of a different class, which caps macro recall at 0.969 for any per-packet model.”

Like you're five: Most notes in web attacks are **empty envelopes** saying “got it” (TCP acknowledgements) sent to the website door (port 80). Only 5–17 of every 100 actually have a message inside. Empty envelopes from different bullies look *exactly* the same. We counted: 2.31% of all notes had an identical twin with a *different* answer. When twins have different answers, no guard can get both right, so even a perfect guard could reach at most 0.969. The clues that tell them apart were in columns the dataset's standard recipe throws away.

“After: each change, measured” (the table)

On the site: “Random” splits individual packets; “flow-grouped” keeps every connection entirely in training or entirely in test.

Clues the guard had	Random acc.	Random F1	Grouped acc.	Grouped F1	Meaning
Original 91	96.38%	0.8868	96.37%	0.8892	Where we started.
+ service port	98.02%	0.9245	98.03%	0.9236	Knowing *which door* helped a lot.
+ content	98.02%	0.9268	98.02%	0.9232	What's inside the letters: little help alone, since few letters have content.
+ flow	99.77%	0.9879	99.75%	0.9808	The whole conversation: the big jump.
+ class weights	99.76%	0.9867	99.72%	0.9828	Rare types count more: steadier across tries.

Like you're five: **Random split** = shuffle all notes and split them into study and exam piles, so notes from the same conversation can land in both. **Flow-grouped split** = whole conversations go to one pile only, so the exam is truly new. The grouped number is the honest one. The note under the table says: after removing exact duplicates there were 1,909,719 notes with the old clues and 1,985,768 with all 130 clues (more notes become different from each other once their contents count). Repeating with three random shuffles (seeds 42/43/44) gave 0.983, 0.983 and 0.985 (grouped) and 0.987 each time (random).

On the site: Bars “Before” vs “Final” per class: Fingerprinting 0.681 → 0.926, Uploading 0.758 → 1.000, DDoS_HTTP 0.762 → 0.969, SQL_injection 0.804 → 1.000, XSS 0.847 → 0.978, Password 0.838 → 0.968, Port_Scanning 0.914 → 0.992, Backdoor 0.947 → 0.981, Vulnerability_scanner 0.973 → 0.997, Normal 0.979 → 1.000, Ransomware 0.948 → 0.950, floods already 1.000.

Like you're five: Each bully type's mark before and after. The weak subjects jumped the most; the easy ones were already perfect.

The eight ideas, one by one

Idea	Status	What it means
1. Behaviour features	done	Group notes into **conversations** (same two devices, same doors; a new conversation after 60 s of silence or after 120 s in total). Every note gets 20 conversation clues: how many notes, how long, how fast, how many “hello/goodbye/hang-up” signals (SYN/FIN/RST), total content, web requests, bad-word totals. Now empty envelopes carry their conversation's story. Biggest jump: 0.923 → 0.981. IP addresses and times are used only to *group*, never as clues.
2. Content as numbers	done	18 number clues about the message: length, how jumbled it is (**entropy**), share of readable letters, whether it's a web request, the web reply code, header counts, web-address length and parts, special characters, and counts of six groups of tell-tale words (database tricks, script tricks, uploads, logins, sneaky folder paths, names of hacking tools). Plus **service port**: which door the conversation uses (80 = website, 1883 = sensor messages, 4321 = what the backdoor uses). The random “return-address” doors are not used, because their numbers follow the recording time, which would give away the answer. The port alone: 0.889 → 0.924.
3. Per-class thresholds	not needed	A trick to tune when to say each answer. The problem it fixed (e.g. Uploading being under-called) disappeared: Uploading is now about 1.00/1.00.
4. Specialist model	not needed	A second guard just for the web family. That family, which held 84% of the old mistakes, is now almost clean; the leftover Password/DDoS_HTTP mix-up is only 2–4%.
5. Tuning settings	partly	We set the guard's study settings sensibly (63 leaves per tree, learning speed 0.1, uses 80% of clues and rows each round, stops when it stops improving for 50 rounds), but didn't search for the best settings. The new clues did the work.
6. Other boosters	not needed	Other tree libraries or mixing several. Not needed: the guard is already near the best possible score for these clues (the “ceiling”, 0.9999).

7. Class weights	done	Rare bully types count a bit more (square-root rule). Without it, the score wobbled between tries (0.981, 0.956, 0.952) because Fingerprinting, with only 156 conversations, scored anywhere from 0.49 to 0.88. With it: steady at 0.983–0.985, Fingerprinting about 0.93. Even studying only 5% of the data: 99.7% and 0.979.
8. Check it's real	done	We used the strict conversation-level exam and three seeds. This caught a real problem: all normal “remote screen” (VNC) traffic was *one* conversation of 264,094 notes. It landed entirely in the exam, so the guard had never seen anything like it and called 16% of normal notes bullies. Cutting long conversations into 2-minute pieces, which is what standard network tools do, fixed it. The old clues scored the same on both exams, so they weren't cheating either.

“Assumptions behind the final number”

- **Per connection record, up to 120 s:** the guard judges a whole conversation piece, not the very first note.
- **The service port is partly a testbed effect:** in this dataset friendly traffic never used the website door, so “door 80” already hints “bully”. In a real office with normal web browsing it would help less.
- **Natural mix:** 71% friendly notes, all notes after removing duplicates, 3 seeds.
- **Now used everywhere:** the whole-school training, every experiment and the live demo all use the 130 clues. Numbers in [brackets] on the site are the old 91-clue results.

7. The pipeline at a glance

On the site: “Two phases, kept deliberately separate: Phase A trains the models offline (federated), and Phase B loads the saved bundle and enforces decisions on live traffic.” Legend colours: Data, Boosting (supervised), Federated clients/autoencoder, Server-side robustness, Network/SDN, Output artefact.

Like you're five: **Phase A** = school term: everyone studies, then the trained guards are packed into a box (the “bundle”). **Phase B** = the guards go on duty at the door. Keeping them apart means study problems and door problems can't get mixed up. The colours on the diagram say which kind of job each box is.

Phase A diagram, box by box

Box	Says	Like you're five
1 Raw dataset	2,219,216 rows × 63 columns, 15 classes	The big pile: 2.2 million notes, 63 facts about each, 15 kinds (normal + 14 bully types).
2 Clean & encode	drop 16 columns, fix label-leak placeholder, dedupe, repair MITM, one-hot; +39 port/ content/flow features → 1,985,768 × 130	Fix the hidden cheat sheet (“0” vs “0.0”), fix the broken MITM rows, add the 39 new clues (which door, what's inside the letter, the whole conversation; name tags and times are used only to group notes into conversations, then thrown away), remove exact copies, and turn words into numbers (“one-hot”: a yes/no column per word). Result: 2.0 million notes × 130 number clues.

3 Split	Server calibration set 5% (labelled); the rest, or in the first runs a 60,000 sample, goes to the classrooms	The principal keeps 5% of the notes, with answers, to teach guard 1. From the rest, 60,000 are given out to classrooms (a smaller set so the 150 test runs finish in time).
4 Boosting bootstrap	LightGBM on raw features, 300 rounds, sqrt class weights; trained before round 1; held-out accuracy 0.997	Guard 1 studies first, so classrooms never use an untrained guard. Rare bully types count a bit more (class weights). On a practice test it got 99.7% (96.3% before).
5 Non-IID partition	Dirichlet α=0.3 → 10 clients; 15% test, 15% benign val; z-score per client, clip ±10	Share notes unevenly among 10 classrooms, as in real life where each site sees different traffic (“non-IID”). Each classroom keeps some notes for exams and some friendly ones for setting its alarm level, and rescales its numbers its own way, with extremes capped at 10.
6 Client round (×10)	boosting filters; train autoencoder; 130→32→16→8; 5 epochs; MSE	Each classroom lets guard 1 remove the obvious bullies, then teaches guard 2 on what's left by squeezing 130 numbers into 8 and rebuilding them, 5 practice passes, scored by how big the rebuilding mistakes are (MSE).
7 Server aggregation	Δ = new − old weights; trust filter; trimmed mean 15% → global AE; ×20 rounds	The principal looks at what each classroom *changed* (Δ), throws out suspicious changes, averages the rest while ignoring the 15% most extreme from each side, and sends back the improved guard 2. 20 times.
8 Boosting update	every 3 rounds: ≤50 confirmed alerts per surviving client; +50 trees or retrain	Every third round, honest classrooms send up to 50 suspicious notes that a human confirmed, and guard 1 learns from them.
9 Phase A bundle	models, per-client scaler & threshold, manifest → saved_models/phase_a	Pack everything into a labelled box: both guards, each classroom's settings, and a list of what's inside and how it was made.

Arrow labels: “63 cols”, “1.99M × 130”, “labelled”, “pool → 60k”, “model broadcast” (guard 1 sent to classrooms), “Δ weights + alerts” (changes sent up), “new global AE ×20” (the loop), “round 20” (then save).

Phase B diagram, box by box

Box	Like you're five
1 Captured traffic	A real recording (a “.pcap” file) of a Backdoor attack: 24,914 notes.
2 Mininet replay	A pretend network with 10 computers and 1 switch; the switch obeys a controller (os-ken, speaking OpenFlow 1.3). We play the recording from computer h1 (tcpreplay).
3 tshark features	A tool (tshark) reads each note and writes down all 61 facts the dataset has, spelled the same way, works out the same 39 extra clues with the very same code as training, then turns words into numbers exactly like in training.
4 Load bundle	Open the box from Phase A. Computer h1 is classroom 0, so it uses classroom 0's settings.
5 Boosting stage	Guard 1 names each note; if it's an attack and guard 1 is ≥ 70% sure, that's the answer.
6 Autoencoder stage	Otherwise guard 2 checks: too different from normal → “anomalous”, else “benign” (friendly).
7 Device verdict	For the whole device: the most common bully label, the average confidence, and which guard decided most.

8 Mitigation bridge	A small web service (REST) gets the verdict. ≥ 85% sure → **block**; ≥ 50% → **rate-limit** (slow down); otherwise allow and watch.
9 Switch flow table	The controller sends the switch a rule (“flow-mod”): drop, or a speed limit (“meter”). We read the switch's rule list (ovs-ofctl dump-flows) to prove it's there.

“The cascade decision rule (one implementation, used by evaluation, dashboard and SDN)”

1. Guard 1 guesses a type and how sure it is.
2. If it's an attack and it's at least 70% sure, that's the final answer; guard 2 isn't asked.
3. Otherwise guard 2 measures how badly it can rebuild the note (the “reconstruction error”). If that's above this classroom's alarm level (“threshold”), the answer is “anomalous” (maybe a brand-new bully), with confidence = how far above the line it is, capped at 1. Otherwise it's “benign”.

Why it matters: The exact same rule is used in testing, on the dashboard and at the door, so the scores we report are the same decisions the door makes.

8. “Every step explained” (the 12 fold-out steps)

On the site: “What each stage does, how it works, and the parameters that control it. Values are the ones used in this project's runs (configs/config.yaml); module paths are relative to fl_ids/.”

Like you're five: Each step opens like a drawer. Inside: **Function** (what it's for), **How it works**, and **Parameters** (the settings knobs, all kept in one settings file, config.yaml). The grey text on the right of each step is the program file that does it.

Step 1: Load, clean and encode the data

- **Function:** turn the dataset file into a clean number table **X** (1,909,719 × 91) and answers **y** (15 types as numbers).
- Read 47 of the 63 columns: drop 15 name-tag/time/raw-message columns as the dataset's authors suggest, plus `Attack_label`, which is just “attack or not” and would hand the guard the answer.
- **Fix the placeholder:** “doesn't apply” was written `0` in some recordings and `0.0` in others, and the spelling matched the answer. One column alone (`mqtt.topic_0.0`) told friendly from bully perfectly (AUROC 1.0 = perfect separation). We made both spellings the same; now the best single clue only reaches about 0.71. We spotted this ourselves, but Galal (arXiv, August 2026) published it first, so the site credits that paper.
- Drop empty rows and 309,497 exact copies.
- **Repair MITM:** the MITM rows had values under the wrong column names. We replaced the 1,214 broken rows with 1,229 rows rebuilt from the original recording. This one is our own find.
- Turn 7 word columns (like the web method “GET”) into yes/no number columns → 91 clues. Parameters list the counts and the 7 columns; if the repaired MITM file is missing, the program stops loudly instead of silently using broken data.

Step 2: Service-port, content and flow features

- **Function:** give guard 1 the clues that separate web attacks. First tested on guard 1 alone, now plugged into the whole system: training, every experiment and the live demo. The same code makes the clues in training and live, and 99.8% of live notes came out exactly equal to their training copies.
- Service port (1 clue), content (18 clues) and flow/conversation (20 clues), as described in section 6. Never used as clues: IP addresses, times, random return doors, row order (each would reveal which recording, and so which answer).
- Class weights and the strict conversation-level exam.
- Parameters: 91 → 130 clues; 1,985,768 notes after removing copies; 60 s / 120 s conversation cut-offs (the 120 s one is the CICFlowMeter tool's default); 63 leaves, learning speed 0.1, stop after 50 rounds without improvement; weight = √(biggest class size ÷ this class size).

Step 3: Split: server calibration set and federated pool

- 5% of notes (same mix of types) go to the principal with answers; 20% of that is kept for testing each new version of guard 1, leaving 79,430 to teach with (76,388 with the old clues). In the first runs the rest was trimmed to 60,000 notes for classrooms (45,474 left to learn from, about 125,000 in all). In the newest runs the classrooms get all the rest, about 1.89 million notes, with each conversation kept together. Only the stand-alone guard 1 in section 6 studied 1.43 million. Answers stay with the principal; classrooms never reveal their notes' answers.

Step 4: Non-IID partition and per-client normalisation

- For each note type, a random “Dirichlet” draw decides how much of it each classroom gets. Small α (0.3) = very uneven; one classroom may be 94% friendly notes.
- Each classroom keeps 15% for its exam and 15% of its friendly notes to set its alarm level; the rest is for study.
- Each classroom rescales numbers using *only its own* study notes (a z-score: how many “typical steps” from average), capped at ±10 so one odd counter can't drown out everything.
- Guard 1 always gets the raw, un-rescaled numbers: each classroom's rescaling is different and would confuse a shared guard.
- Note: classroom 5 got no friendly practice notes, so its alarm never rings. We report this rather than hide it.

Step 5: Boosting bootstrap (cold start)

- Guard 1 studies before round 1, on raw numbers, adding one tree per type per round (15 trees per round). Settings: 300 rounds, learning speed 0.05, 31 leaves. **Leaf regularisation** (min_sum_hessian, lambda_l2) keeps it calm; without it, training went wild and false alarms jumped between 0.1% and 16% when just 1% of the notes changed. “Deterministic, one thread” = same answer every time. Rare types get a bit more weight (√ class weights). Practice-test accuracy: 0.997 (was 0.963).

Step 6: Client round: boosting filter, then autoencoder training

- Each classroom removes notes guard 1 confidently calls bullies, then trains guard 2 only on the rest, so guard 2 never learns bullies as “normal”.

- Guard 2's shape: 130 → 32 → 16 → **8** → 16 → 32 → 130 (9,834 knobs; 7,299 with the old 91 clues). It squeezes and rebuilds; mistakes are measured with MSE.
- It sends back its knob changes, a 50-bar chart of its rebuilding errors and, every third round, up to 50 confirmed suspicious notes. Otherwise notes never leave.
- Alarm level = the 97th percentile of rebuilding error on its friendly practice notes: the level that only 3% of friendly notes exceed. So about 3% false alarms, by design.
- Settings: learning speed 0.001, 5 passes, 64 notes at a time; 97th percentile; 0.7 confidence for guard 1.

Step 7: Server aggregation: trust filter + trimmed mean

- The principal compares **changes**, not whole brains: whole brains all look about 99% alike because they start from the same copy.
- The reference is the **median** change (the middle value for every knob). For each classroom it measures the **cosine similarity** (do the changes point the same way? +1 = same, 0 = unrelated, −1 = opposite) and the size.
- A classroom is excluded if (a) it is an outlier compared with the others this round (MAD test), (b) its change points the opposite way (cosine < 0), or (c) its long-term trust score has fallen below 0.5.
- Remaining changes are size-capped at 2× the typical size and averaged with a trimmed mean (drop the top and bottom 15% for each knob). If nobody passes, the round is skipped.
- Settings: MAD 3.0; minimum cosine 0; minimum trust 0.5; trust memory 0.3 (30% this round, 70% history); cap 2.0; trim 0.15.
- **Known limits:** at 50% sneaky classrooms, the “middle” becomes theirs; late in training, sneaky changes can look like honest noise.

Step 8: Incremental boosting update

- Every 3 rounds, honest classrooms send up to 50 confirmed suspicious notes (false alarms come back as “Normal”). Guard 1 adds 50 trees per type. If a note is a *brand-new* type, guard 1 is retrained from scratch, because adding trees can't teach a type it has never seen. The new guard 1 is sent out next round. Trade-off: those few confirmed notes do leave the classroom. This is also the slowest step.

Step 9: The Phase A bundle

- The real run (1 principal + 10 classroom programs, 20 rounds) saves guard 1, guard 2's knobs, each classroom's rescaling and alarm level, and a manifest (settings, scores, seed, code version). Before saving, it recomputes the alarm levels and checks they match; a mismatched box is rejected. Settings: 20 rounds; principal's address 127.0.0.1:9091; option to add sneaky classrooms for demos.

Step 10: Replay and feature extraction (Phase B)

- Mininet builds 10 computers on one Open vSwitch with the os-ken controller; computer h(i+1) is classroom i. tcpreplay plays the recording; tshark extracts all 61 facts per note, and the same code as training adds the 39 extra clues. Newer tshark writes True/False and short hex numbers, which silently zeroed one clue until we fixed it. Mininet needs administrator rights, given by one narrow permission rule.

Step 11: Verdict, mitigation bridge and flow rule

- The device's verdict is its most common bully label (or benign). It goes to the bridge, which asks the controller for a **drop** rule (≥ 0.85) or a **meter**/speed limit (≥ 0.5). We check the switch's own rule list, not just the bridge's word. Ports: 6653 for the controller, 8080 for the bridge. Limitation: it needs the whole recording first, so it's not note-by-note.

Step 12: Evaluation harness and live dashboard

- **Variants** compared fairly: full system; plain averaging (FedAvg, no trust filter); trimmed mean only; guard 2 only; guard 1 only.
- Headline scores are not plain accuracy, because 71% of notes are friendly, so a broken guard that never shouts still looks “accurate”.
- **Poisoning sweep** at 0/20/30/40/50% sneaky classrooms; **zero-day** test hiding each of 14 types.
- Tests run all classrooms inside one program to save time; the real multi-program version is Phase A.
- The **Streamlit dashboard** is a live web page showing the training and the door as they happen.

9. “Does the pipeline work as intended?”

On the site: “Yes — each part was verified on real data and real processes, not just unit tests.”

Item	Like you're five
Ablation	“Ablation” = remove one part and see what breaks. With 30% sneaky classrooms: the full system catches 1.000, guard 1 alone 0.996, guard 2 alone 0.372 [before: 0.887 / 0.875 / 0.191]. For brand-new bullies: 0.930 vs 0.795 [0.731 vs 0.518]. So guard 2 matters mostly for *new* bullies. On bullies it already knows, guard 1 alone now has the better score (0.998 vs 0.961), because guard 2's planned 3% false alarms cost more than the little it adds.
Poisoning resistance	Catch rate stays between 0.997 and 1.000 from 0% to 50% sneaky classrooms [0.878–0.887], and new-bully catch stays 0.90–0.94 up to 40%. Sneaky changes that got past the filter: 10% / 12% / 36% at 20 / 30 / 40% [28% / 32% / 24%]. At 50% the filter breaks: every sneaky change gets through. That's expected.
Zero-day detection	Each of 14 types hidden in turn. Both guards together: 0.943 [0.736]; guard 1 alone 0.801 [0.533]. Big gains: Password 0.44 → 0.97, SQL_injection 0.61 → 1.00, XSS 0.52 → 0.99. Where guard 1 knew nothing, guard 2 saved the day: Uploading 8% → 72%, MITM 0% → 69%. Two got worse than before: MITM (was 89%) and Uploading for guard 1 alone.
Real federated training	11 real programs, 20 rounds: 0.999 catch rate [0.889] with 3.5% false alarms, matching the faster one-program simulation.
Live SDN mitigation	Real attack replayed → “Backdoor”, 99.7% sure → drop rule installed and seen on the switch.
Classifier accuracy ≥ 95%	99.7% and 0.983 on unseen conversations; 0.983–0.985 over three tries; 99.7%/0.979 studying only 5% of the data.
Live dashboard	We watched it in a browser during real training, and found and fixed a start-up bug that the automatic tests had missed.

On the site: “Poisoning-resistance sweep” chart: x-axis = % of clients running a sign-flip attack; lines = attack macro-recall, zero-day macro-recall, AE-alone macro-recall. “AE-alone and zero-day recall collapse together at 50%.”

Like you're five: Left to right: more and more sneaky classrooms. A **sign-flip attack** is a sneaky classroom sending the exact opposite of what it learned, five times louder. The top line (catching known bullies) stays flat; the middle line (new bullies) stays flat until 40%; the bottom line (guard 2 alone) drops to 0 at 50%, when the sneaky ones become the majority.

“Live SDN demo: what actually happened”

On the site: An earlier demo misclassified this capture as DDoS_HTTP; after fixing two data bugs (the label-leak and a tshark version issue) and using a fully trained bundle, it classifies correctly.

Row	Like you're five
Capture	The Backdoor recording: 24,914 notes, 24,669 judged not-friendly (23,895 before the new clues).
Classification	“Backdoor”, 99.7% sure, decided by guard 1 alone.
Mitigation action	Block.
Installed flow rule	`priority=100,ip,nw_src=10.0.0.1 actions=drop` = “high-priority rule: any note from computer 10.0.0.1, throw it away.”
Verified against	The switch's own rule list.

Careful: “Real limitation”: 23,161 notes had already passed through the normal “allow” rule before the block rule appeared, because the system judges after reading the whole recording. The door locked, but late. This is how it's built today, not a bug, and the site says so.

10. Caveats and limitations (all 16)

On the site: “Every number above is real, but every result has a scope.”

1. **Mitigation is not real-time:** the door locks after the recording is analysed.
2. **The DL contest used a rebalanced sample:** friendly notes were cut from 71% to 10%, so its false-alarm numbers (16%, 53%) can't be compared with the system's normal ones (0.02%). Who won is still fair.
3. **Different experiments' F1 can't be compared:** 0.887 (contest), 0.910 (small federated test, where MITM had 1 test note and Fingerprinting 3), 0.983 (final). Compare only within one experiment.
4. **Random-split optimism, now checked:** the strict and random exams agree within about 0.005. Splits by whole recording file or by time weren't tried.
5. **Service port is partly a testbed effect** (no friendly website traffic in this dataset).
6. **Flow clues describe a whole conversation piece (≤ 120 s):** not a first-note decision.
7. **Trade-offs of the new clues:** on bullies it already knows, guard 1 alone now scores better than both guards together (0.998 vs 0.961) because of guard 2's planned 3% false alarms; two hidden bully

types got harder to catch (MITM 0.89 → 0.69, and Uploading for guard 1 alone); and at 40% sneaky classrooms more sneaky changes got through (24% → 36%, one seed only).

8. **MITM can't be scored on the strict exam:** it's one conversation (1,229 notes). On the random exam its F1 is 0.994.
9. **Ideas 3–6 weren't run:** no claims about them.
10. **Unusual but honest classrooms can look sneaky:** a classroom with very unusual notes sometimes gets excluded for a round.
11. **The filter breaks at 50%;** trimmed mean alone also breaks once sneaky classrooms outnumber what it trims.
12. **Two kinds of sneaky classroom tested** (sign-flip, and a clever one that knows the filter and mostly slips past it); other clever ones are untested.
13. **Whole-school tests now use three seeds** for the main experiments (ablation, sweep, new-bully test); the pruning and clever-attacker tests use one. One odd pattern (20% vs 30%) disappeared with other seeds, so it was noise.
14. **The whole-school system now uses all the notes** (about 1.89 million shared out, conversations kept together; see the Cumulative page, 7d), but guard 1 inside it still learns from the principal's 5% (about 72–80 thousand notes). The first 150 or so test runs used a 60,000-note sample, and those results are kept for comparison.
15. **Tiny classes:** MITM (358–406 notes) and Fingerprinting (853) give few exam notes, so their scores wobble.
16. **Novelty isn't proven:** our searches found no system with all five parts, but that's not a complete review.

11. The Cumulative results page

On the site: “Every experiment run for this project, in one place … All values are loaded directly from the result files … nothing is retyped. Best value in a column is highlighted green; clear weaknesses red.”

Like you're five: This page is the big report-card binder. The computer fills the tables straight from the result files, so nobody could mistype a number. Green = best in that column; red = clearly weak.

Run provenance (“where these numbers came from”)

Row	Like you're five
Dataset	2,219,216 notes → 1,985,768 after cleaning, 130 clues, 15 types. Old 91-clue runs (section 9 and the [brackets]): 1,909,719 notes. (The DL contest had 1,909,671 because it ran before the MITM fix.)
Classifier-accuracy runs	Section 8: 130 clues, 1,985,768 notes, natural mix, random and strict exams.
Seed	The random “shuffle number”: 42 for everything, 43 and 44 for repeat checks. Same seed = same shuffle = repeatable.
Federation	10 classrooms, unevenness 0.3, 20 rounds, 60,000 shared-out notes, principal keeps 5%.

Hardware	MacBook Pro M5 running an Ubuntu pretend-computer: 4 cores, 7.2 GB memory (12 GB for the real run), no graphics card.
Software	Python and libraries with versions; 233 tests pass, 2 skipped.

Section 1: Boosting vs. supervised DL classifier

Like you're five: The same contest as on the main page, plus a per-type table with **precision** (“when you say this type, how often are you right?”), **recall** (“of this type, how many did you catch?”), **F1** (both combined) for each guard, and **F1 gap** (boosting minus DL). Stand-out rows: XSS gap +0.724 (DL caught only 7%); Fingerprinting DL 0.000 (never right); Password DL precision 0.282 (when DL says “Password” it's usually wrong). “Test rows” = how many exam notes of that type (e.g. 9,848 for most; only 72 for MITM and 171 for Fingerprinting).

Section 2: Per-class report (full federated system)

Like you're five: One full-system training with no sneaky classrooms, examined on 9,000 exam notes from the classrooms. **AUROC** = how well a guard ranks bullies above friendly notes (1.0 = perfect, 0.5 = coin flip). Guard 1 alone: accuracy 0.997, macro F1 0.978 [0.963 / 0.910]; perfect on floods, Uploading, SQL_injection and XSS; weakest is Ransomware (recall 0.879, only 58 exam notes) [before: Uploading 0.572, DDoS_HTTP 0.628, SQL_injection 0.706]. MITM (3 exam notes) and Fingerprinting (3) are too few to trust. Second table, “friendly vs attack” only: guard 2 alone catches 73% of attacks [50%] with 3.3% false alarms, and ranks bullies above friendly notes much better (AUROC 0.982 [0.805]); both guards together (“cascade_combined”) catch 99.95% of attacks [88.3%] with overall accuracy 0.976 [0.944].

Section 3: Ablation

Like you're five: Five versions of the system, each with 30% sneaky classrooms (classrooms 0, 6 and 9). Columns: **attack macro-recall** (catch rate averaged over bully types), **benign FPR** (false alarms), **detection F1** (friendly-vs-attack score), **zero-day macro-recall** (new bullies), **AE-alone recall** (guard 2 by itself), **final val loss** (guard 2's final rebuilding mistake; small = healthy), **rounds to converge** (when learning settled), **comm. bytes** (data sent: 15.7 million bytes, about 15 MB, over 20 rounds; 11.7 million with the old 91 clues, because guard 2 now has more knobs).

- **full_pipeline:** catches every known bully (1.000), best new-bully catch (0.930 [0.731]), guard 2 still works (0.712 [0.421]), healthy loss 0.028, settled by round 5.
- **plain_fedavg** (no filter, plain average): guard 2 ruined (0.002), loss exploded to 5×10^{20}, never settled. New bullies fall back to guard 1’s 0.79.
- **trimmed_mean_only:** also ruined (0.000), loss 13,162.
- **autoencoder_only** (no guard 1): catches only 0.372 [0.191].
- **boosting_only:** 0.996 [0.875], almost zero false alarms (0.02%), best friendly-vs-attack score (0.998), but only 0.795 on new bullies [0.518]. No federated training, so 0 bytes sent.

The second table lists each bully type's catch rate under each version. In the full system every type is caught (1.000). Guard 2 alone is weak on most (e.g. Backdoor 0.02) but perfect on MITM (1.000).

Section 4: Poisoning-resistance sweep

Like you're five: The full system at 0%, 20%, 30%, 40% and 50% sneaky classrooms. **Attacker survival** = how often a sneaky classroom's change got past the filter: 0.100, 0.117, 0.362, then 1.000 at 50% (the filter fails) [before: 0.275, 0.317, 0.238, 0.960]. Final loss stays tiny until 50%, where it explodes (3.8×10^{18}). Known-bully catch stays 0.997–1.000 at every level.

Section 5: Multi-seed check

Like you're five: With the old clues, guard 2 oddly did *better* at 30% sneaky classrooms than at 20% at seed 42; seeds 43 and 44 showed that was luck (noise). We re-ran the check with the new clues: guard 2 alone scored 0.736 / 0.712 (20% / 30%) at seed 42, 0.738 / 0.734 at seed 43 and 0.623 / 0.565 at seed 44. So guard 2 alone still wobbles between seeds (0.56–0.74, was 0.19–0.42), while the main catch rate is rock-steady (0.996–0.999, was 0.83–0.89).

Section 6: Zero-day (leave-one-attack-class-out)

Like you're five: For each of 14 bully types, hide it completely (even from the rescaling), train, then show up to 5,000 of its notes. Columns: **Boosting-only** (guard 1's catch rate), **AE-alone**, **Cascade** (both), **AE added** (extra caught thanks to guard 2), **Boosting's top misattribution** (what guard 1 thought the new bully was), **AE AUROC**, **Cascade FPR** (about 3% false alarms everywhere). Examples: Uploading guard 1 0.080 → both 0.716 (guard 2 added 0.636); DDoS_UDP 0.718 → 0.983; Backdoor mostly mistaken for Ransomware (99%); DDoS_ICMP mistaken for Fingerprinting (100%), but still caught as “some attack”; hardest to catch as new: MITM 0.695 and Uploading 0.716 [before: DDoS_HTTP 0.360, Password 0.443]. Guard 2’s AUROC is now 0.97–1.00 for every type.

Guard 1 alone (blue) vs both guards (teal) on each hidden bully type.

Section 7: Real federated training and live SDN demo

Like you're five: Four real-world checks: the real 11-program run (0.999 [0.889], 3.5% false alarms, guard 1 practice accuracy 0.997 [0.963], about 4 min); the live door demo (Backdoor, 0.997, drop rule verified; 24,669 notes judged not-friendly; 23,161 notes passed before the rule); the dashboard during a real run (round 20, all 10 classrooms trusted, guard 1 at version 7; watched during the old 91-clue run); and the test suite (233 passed, 2 skipped because two recordings weren't downloaded).

Section 8: Classifier accuracy (final)

Like you're five: Every guard-1 experiment, one per row. Columns: **features/setting** (which clues; “500,000-row sample” = the contest-style set; “trained on 5% of rows” = only the principal-sized set; “class weights” = rare types count more), **split** (random or strict), **seed**, **features** (how many clues), **test rows**, **accuracy**, **macro F1**, **ambiguous rows** (share of notes with a twin that has a different answer: 2.31% → 0.29% → 0.01%) and **ceiling macro recall** (the best any guard could do with those clues: 0.9692 → 0.9940 → 0.9997). Green = at least 0.95, red = below 0.9. The rows show the unweighted wobble (0.9808/0.9560/0.9521) and the weighted steadiness (0.9828/0.9835/0.9851).

Like you're five: The last table, “Final classifier, per class”: each type's precision, recall and F1 now, next to its F1 with the original clues. For example SQL_injection 0.804 → 1.000, Uploading 0.758 → 1.000, Fingerprinting 0.681 → 0.926.

Section 9: Original 91-feature federated results (for comparison)

Like you're five: Sections 2–7 now show the system with the 130 clues. Section 9 keeps the *old* tables, made with the original 91 clues, so you can see the before-and-after side by side. These are the numbers shown in [brackets] everywhere else on the site.

Index: what each result is good for, and its limits

Result	Proves	Does NOT prove
Boosting vs DL	a same-effort DL guard is worse here	that no DL guard could win; natural-mix results
Ablation	each part helps; guard 2 mostly helps with new bullies	that it holds across seeds
Poisoning sweep	works up to 40% sign-flip sneaks	cleverer sneaks; above 50%
Zero-day	guard 2 catches types guard 1 never saw	bullies unlike all 14 known types
Classifier accuracy	≥ 95% and ≥ 0.95 on unseen conversations, steadily	networks with friendly web traffic; first-note (instead of whole-conversation) decisions
Real Phase A/B	it all runs end to end on real programs and a real switch	note-by-note, real-time blocking

Why it matters: This index is the most honest table on the site: for each result, it says what you're allowed to claim and what you're not.

11b. What was added after this guide was first written

On the site: “Real runs, repeated over seeds” · “Follow-up experiments” (on the Cumulative results page, sections 7b and 7c) · “Scaling to 20 clients” and “The whole journey, timed” (on the main page).

- **Scaling to 20 clients.** The principal now also runs with 20 schools. Before, this was pretend; it has been run for real (one server plus 20 separate programs), on a computer with enough memory. Attack detection stays about the same; the table lists every run.
- **The whole journey, timed.** From raw notes to a locked door took about 219 seconds in one run: most of it is learning together, and the live part (replay, spot, lock) takes seconds.
- **Real runs, three seeds (7b).** Each size was run three times with different shuffles. The one low score (0.978) is one missed note out of a group of 4.
- **Follow-up experiments (7c).** The big tests were repeated with two more seeds (same story); a clever saboteur mostly slips past the filter; deleting 70% of the numbers in guard 2's brain works if done while learning, not after; and a stricter “earlier teaches, later tests” exam gives 99.4% accuracy.
- **The whole dataset (7d).** Everything was run again with all the notes and with whole conversations kept together on one side of the exam. Known bullies: still about 99.6% caught. False alarms look worse note by note (2.5–16%) than conversation by conversation (under 1%), because a few long friendly conversations get flagged as a block. New bullies: 90–94% (was 94–95%). All 14 recordings are blocked in the demo; 12 are named correctly.
- **What a table cell means.** “0.745 / 0.727 / 0.688” style cells list the three seeds in order (42 / 43 / 44) so you can see how much they differ.

Like you're five: When something still wobbles after three tries, we show you all three numbers instead of hiding it behind one.

12. Dictionary: the 15 kinds of traffic

Type	Like you're five
Normal	Everyday friendly messages: sensors reporting temperature, devices chatting (MQTT), remote-screen (VNC).
DDoS_UDP	A crowd throwing thousands of balls at one window so it can't open for anyone else.
DDoS_ICMP	A crowd shouting “are you there?” (ping) nonstop until the device is too busy to answer anyone.
DDoS_TCP	A crowd starting thousands of conversations and never finishing them (SYN flood), so no chairs are left.
DDoS_HTTP	A crowd asking the website for its front page again and again until it's too tired to serve real visitors.
SQL_injection	Typing secret database commands into a website's form box to trick it into revealing or changing data.
XSS	Hiding a little script inside a web page so other visitors' browsers run the bully's code.
Uploading	Sneaking a harmful file (like “hack.php”) onto the website through its upload button.

Password	Guessing passwords over and over until one works (brute force).
Backdoor	A secret door program left on a device so the bully can come back anytime.
Ransomware	Locking up someone's files and demanding money to unlock them.
Vulnerability_scanner	A tool that checks every corner of a website for weak spots.
Port_Scanning	Knocking on every door number to see which ones open.
Fingerprinting	Poking a device in special ways to figure out what kind it is and what software it runs.
MITM	Man-in-the-middle: secretly standing between two people and passing (or changing) their notes, here by faking addresses (ARP spoofing) and name look-ups (DNS).

13. Dictionary: every score and word

Word	Like you're five
Packet	One note on the network.
Flow / connection	All the notes of one conversation between two devices.
Feature	One clue about a note (its length, its door number…).
Class / label	The right answer for a note (“Normal”, “XSS”…).
Accuracy	Right answers ÷ all answers.
Precision	When the guard says “XSS”, how often it really is XSS.
Recall	Of all real XSS notes, how many the guard caught.
F1	One score that is high only if precision *and* recall are both high.
Macro F1 / macro-recall	The plain average over all types; rare types count as much as big ones.
Weighted F1	Average where bigger types count more.
Micro recall	Catch rate counting every bully note equally (big types dominate).
Attack macro-recall	Catch rate averaged over the 14 bully types.
Zero-day macro-recall	Catch rate on bully types hidden from training.
Benign FPR	False alarms: share of friendly notes called bullies.
Detection F1	F1 for the simpler question “friendly or attack?”.
AUROC	How well a guard ranks bullies above friendly notes: 1 = perfect, 0.5 = coin flip.
Confusion matrix	Table of real type vs guessed type; the diagonal is “right”.
Ambiguity ceiling	The best possible score when some notes are identical twins with different answers.
Reconstruction error / MSE	How badly guard 2 rebuilds a note; big = weird.
Threshold (97th percentile)	Guard 2's alarm level, set so only 3% of friendly notes exceed it.
Validation loss	Guard 2's mistake size on practice notes; small = healthy, huge = broken.
Rounds to converge	The round when learning settled down.
Seed	The shuffle number; same seed = same shuffle.

Random vs flow-grouped split	Shuffle notes, or keep whole conversations together; grouped is stricter.
Supervised / unsupervised	Learning from labelled answers (guard 1) vs only from what normal looks like (guard 2).
LightGBM / boosting	Hundreds of tiny decision trees whose votes are added up.
Deep learning / MLP	A network of layers of adjustable knobs.
Autoencoder	Squeeze-and-rebuild network; learns “normal”.
Federated learning (Flower)	Classrooms share lessons, not notes; Flower is the software that runs it.
Client / server	Classroom / principal.
Non-IID, Dirichlet α	Classrooms get very different notes; α controls how different (smaller = more).
Poisoning, sign-flip	Sneaky classrooms sending harmful lessons; sign-flip = sending the exact opposite, louder.
Cosine similarity, median, MAD	Do two changes point the same way?; the middle value; how spread out values are.
Trust score (EMA)	A memory of how trustworthy each classroom has been (30% today, 70% history).
Trimmed mean	Average after dropping the most extreme values at both ends.
Cascade	Checks in a row: guard 1, then guard 2.
SDN, OpenFlow, Open vSwitch, os-ken	Software-controlled network; the language for giving switches rules; the software switch; the controller program.
Mininet, tcpreplay, tshark	Pretend network on one computer; replays recordings; reads notes and writes down their facts.
Flow rule, drop, meter	A switch instruction; throw away; speed limit.
Phase A / Phase B	Training term / on duty at the door.
Bundle / manifest	The packed box of trained guards / its label listing what's inside.
VM	A pretend computer running inside the real one.